

A GPU-Accelerated Lattice Boltzmann Solver

D2Q9, Kokkos, and distributed-memory MPI

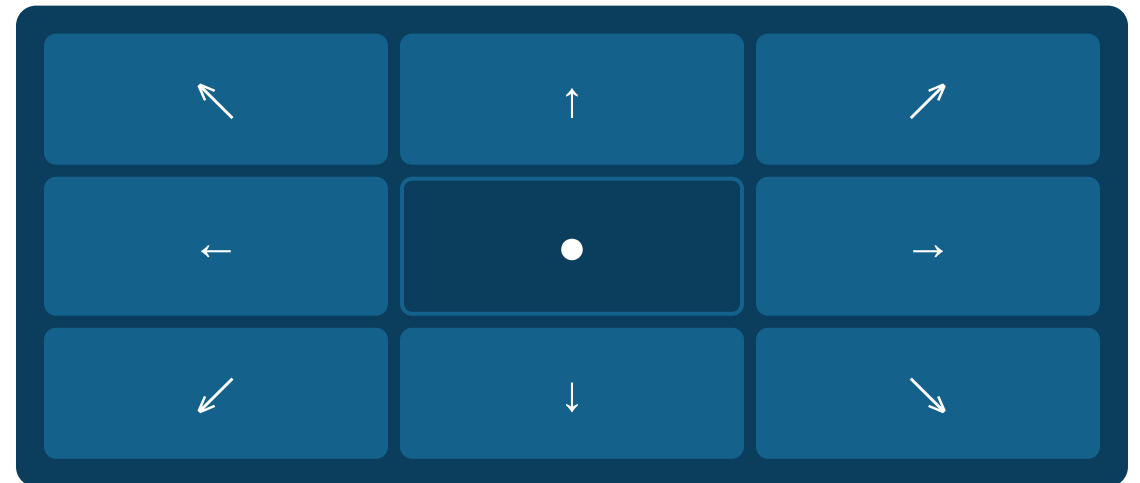
Julius Fischer

HPC with Accelerators — Colloquium, Summer 2026

What is Lattice Boltzmann?

- Simulates fluid flow by tracking **particle distribution functions** $f_i(\mathbf{x}, t)$ on a fixed lattice, instead of solving Navier–Stokes directly
- Each timestep: particles **stream** along lattice directions, then **collide** toward local equilibrium
- Macroscopic ρ , $\mathbf{u}$ recovered as moments of f_i
- **Local, data-parallel** → great fit for GPUs + MPI

D2Q9 lattice — 9 velocity directions



weights: $\frac{4}{9}$ (rest), $\frac{1}{9}$ (axes), $\frac{1}{36}$ (diag.)

Algorithm: Stream → Collide → Bounce-back

Every timestep, for every lattice node:

1. **Streaming** — propagate f_i to the neighbor along direction i
2. **Bounce-back** — enforce solid-wall / moving-lid boundaries
3. **Collision (BGK)** — relax toward equilibrium:

$$f_i(\mathbf{x}, t + 1) = f_i(\mathbf{x}, t) + \omega [f_i^{eq}(\mathbf{x}, t) - f_i(\mathbf{x}, t)]$$
$$f_i^{eq} = w_i \rho \left(1 + 3 \mathbf{e}_i \cdot \mathbf{u} + \frac{9}{2} (\mathbf{e}_i \cdot \mathbf{u})^2 - \frac{3}{2} |\mathbf{u}|^2 \right)$$

Kinematic viscosity is set entirely by the relaxation rate: $\nu = \frac{1}{3} \left(\frac{1}{\omega} - \frac{1}{2} \right)$

Implemented in `src/boltzman.cpp`: `streaming()`, `bounce_back()`, `collision()` / `calc_feq()` in `src/direction.hpp`.

Kokkos-based implementation

- `BoltzmanLattice` owns state as **Kokkos Views**: `distribution`, `buffer` (double-buffered stream target), `density`, `avg_velocity`
- Every update is a `Kokkos::parallel_for` over an `MDRangePolicy` — same source compiles to OpenMP (CPU) or CUDA (GPU)
- `collision_fused()`: density, velocity, f^{eq} , BGK relaxation in *one* kernel instead of four — fewer global-memory round-trips

```
parallel_for
collision_fused all_nodes_policy
KOKKOS_LAMBDA const int const int
// load f_i, reduce → rho, u
// build f_i^eq, relax, write back
// all in registers, one pass
```

Build: `cmake -B build-cuda -DYALB_ENABLE_CUDA=ON`
`-DKokkos_ARCH_<GPU ≥ ON`, same `src/` for CPU and GPU targets.

Correctness by construction

Before trusting performance numbers, the solver has to be *right*.

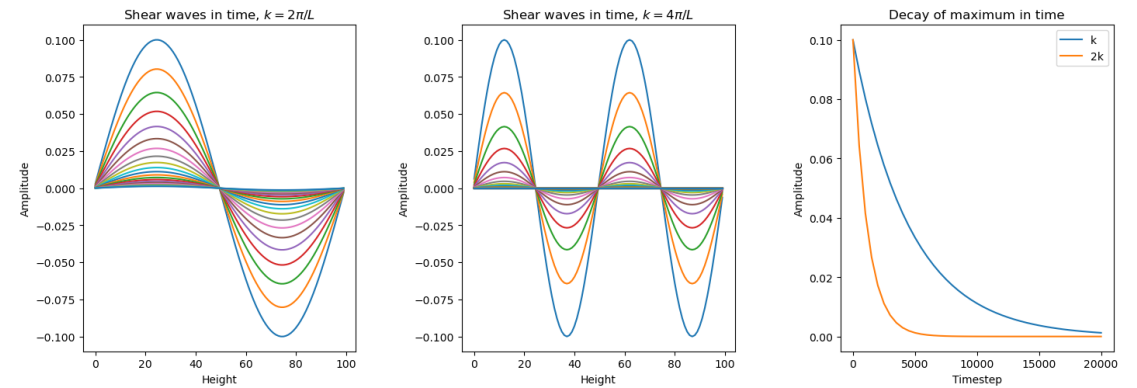
Unit tests (`tests/test_milestone03.cpp` , GoogleTest, run via `ctest`):

Test	What it checks
<code>MassConservation</code>	$\sum_i f_i$ summed over the whole lattice is invariant across 100 stream+collide steps on a randomized initial distribution
<code>MomentumConservation</code>	Total momentum $\sum \mathbf{e}_i f_i$ is likewise invariant

Both pass to within `1e-5` — the discrete Boltzmann update conserves mass and momentum exactly, as the continuous equations require.

Validation: shear-wave decay

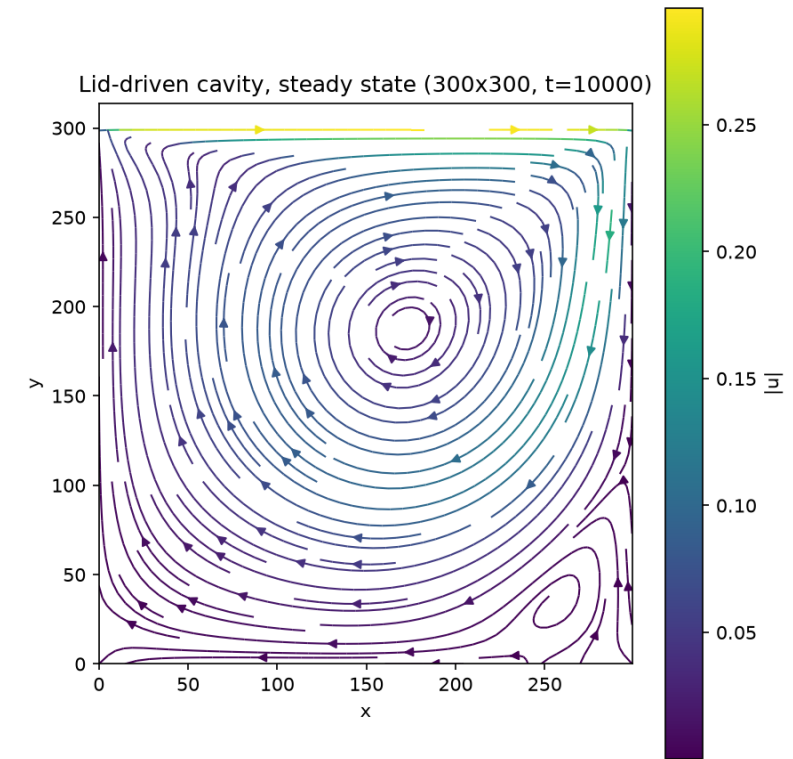
- Initialize a sinusoidal velocity perturbation $u_x(y) = \varepsilon \sin(ky)$, no forcing, and let it decay
- Analytically, the amplitude decays as $e^{-\nu k^2 t}$ — a direct, independent measurement of the solver's viscosity
- Measured decay of the simulated amplitude (right panel, two wavenumbers k and $2k$) matches the analytic prediction from ω



Milestone 04 · `shear_wave_init()` in `src/boltzman.cpp`

Validation: lid-driven cavity

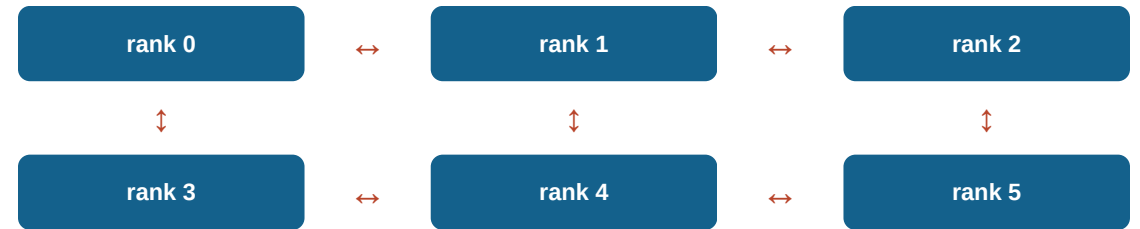
- Classic LBM benchmark: no-slip walls on 3 sides (bounce-back), moving lid on top
- 300×300 grid, $Re = \frac{u_{lid}L}{\nu} \approx 540$, steady state by $t = 10,000$
- Reproduces the expected topology: **primary vortex** at center + **secondary corner vortex** — only appears once solver & BCs are correct



Milestone 05 · `bounce_back()` : no-slip + moving-lid BCs, `src/boltzman.cpp`

Scaling out: MPI domain decomposition

- 2D **Cartesian rank topology**: `MPI_Dims_create` + `MPI_Cart_create`; each rank owns a `size_x/dims[0] × size_y/dims[1]` sub-lattice
- 1-cell **ghost layer** exchanged with all 4 neighbors every step via `MPI_Cart_shift` + `MPI_Sendrecv`
- Boundary slices packed/unpacked by a Kokkos kernel into small staging buffers (a fixed-index slice of the distribution view is non-contiguous under GPU layout)



1-cell halo exchanged with all 4 neighbors, every step.

One rank per GPU. Staging buffers allocated *once*, reused every step — an earlier version re-allocated 8 device views per exchange (~160k `cudaMalloc` / `cudaFree` pairs over a 10k-step run), capping scaling.

Performance & scaling

- Metric: **MLUPS** — Million Lattice Updates/s = $\frac{X \cdot Y \cdot N_{steps}}{t_{run} \times 10^6}$

Grid	Timesteps	Runtime	GPUs	MLUPS
3000 × 3000	10,000	8.28 s	12 (3 nodes × 4 A100)	495.4

- `scaling_tests.jobs` sweeps **strong scaling** (fixed 3000², GPUs ∈ {1, 2, 4, 8, 16}) and **weak scaling** (constant 750²/GPU, same counts)

Scaling sweep queued but not yet completed on the cluster — strong/weak scaling plots go here once `scaling_tests.jobs` finishes running.

Expected: near-linear weak scaling (halo is thin relative to a 750×750 subdomain); strong scaling rolls off once per-GPU work drops below what's needed to hide ghost-exchange latency.

Summary & outlook

What's built: a D2Q9 Kokkos solver — streaming, BGK collision, bounce-back / moving-lid BCs — validated by conservation tests and shear-wave decay, distributed with MPI Cartesian ghost exchange, one rank per GPU.

What it does correctly: mass & momentum conservation to 10^{-5} ; measured viscosity matches shear-wave theory; lid-driven cavity reproduces the expected primary + secondary vortex topology.

Measured so far: 495 MLUPS on 12 A100 GPUs at 3000^2 .

Next: finish the strong/weak scaling sweep; profile the ghost-exchange vs. compute overlap at high rank counts.

Questions?